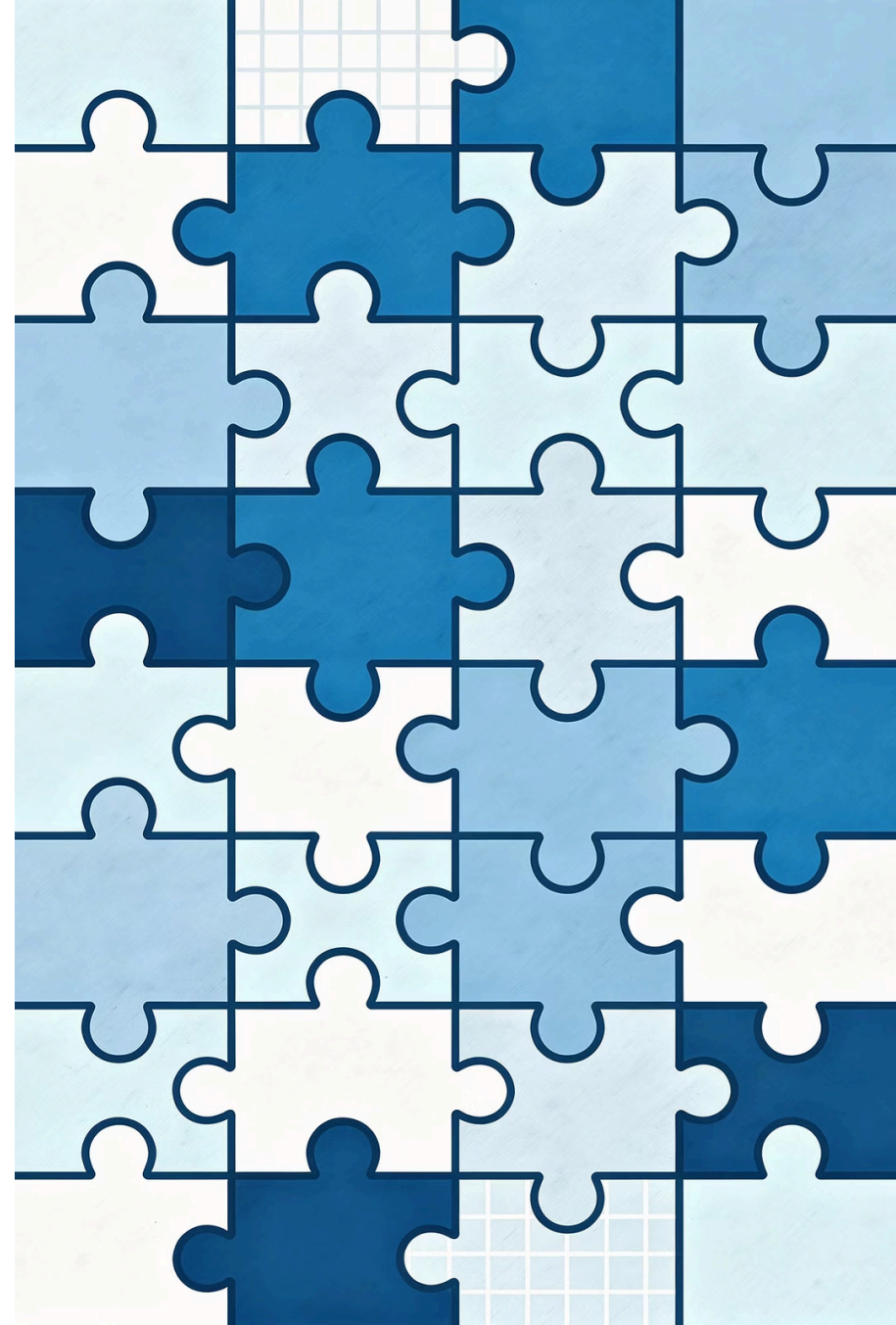


Patrón de Diseño: Adapter

También conocido como: **Adaptador, Envoltorio, Wrapper**

Un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles.



Nuestro Recorrido: Patrón Adapter

Hemos explorado los conceptos clave para entender a fondo este patrón de diseño.

Introducción y Problema

Definición del Adapter, el reto de las interfaces incompatibles y ejemplos iniciales.

Estructura y Tipos

Detalles de la estructura del patrón y el adaptador de objetos (Object Adapter).

Aplicabilidad y Análisis

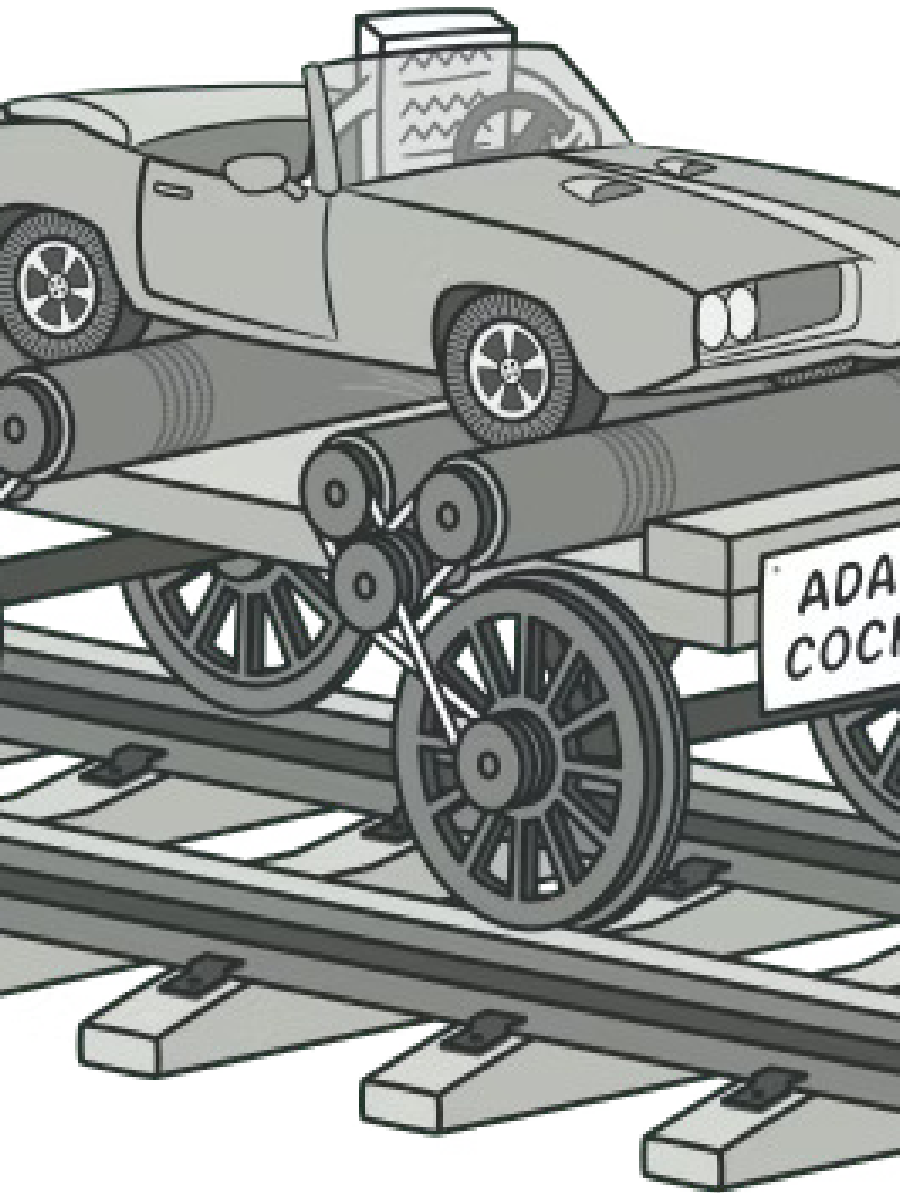
Casos de uso, pros y contras, y sus relaciones con otros patrones de diseño.

La Solución y Analogía

Cómo el adaptador resuelve la incompatibilidad, su funcionamiento y un ejemplo real.

Implementación Práctica

Vistas al pseudocódigo para las clases base, incompatible y la clase adaptadora.



INTRODUCCIÓN

¿Qué es el patrón Adapter?

Adapter es un **patrón de diseño estructural** que permite la colaboración entre objetos con interfaces incompatibles. Actúa como un puente entre dos interfaces que, de otro modo, no podrían comunicarse entre sí.

El Problema

Imagina que estás creando una **aplicación de monitoreo del mercado de valores**. La aplicación descarga la información de bolsa desde varias fuentes en formato **XML** para presentarla al usuario con bonitos gráficos y diagramas.

En cierto momento, decides mejorar la aplicación integrando una inteligente **biblioteca de análisis de una tercera persona**. Pero hay una trampa: la biblioteca de análisis solo funciona con datos en formato **JSON**.



Aplicación

Clases Centrales



Formatos Incompatibles

No puedes utilizar la biblioteca de análisis "tal cual" porque ésta espera los datos en un formato incompatible con tu aplicación.

Opción 1: Modificar la biblioteca

Podría descomponer código existente que depende de ella.

Opción 2: Sin acceso al código fuente

Podrías no tener acceso al código fuente, haciendo imposible esta solución.

La Solución: El Adaptador

Puedes crear un **adaptador**: un objeto especial que convierte la interfaz de un objeto, de forma que otro objeto pueda comprenderla.

Un adaptador **envuelve** uno de los objetos para esconder la complejidad de la conversión que tiene lugar tras bambalinas. El objeto envuelto ni siquiera es consciente de la existencia del adaptador.

Por ejemplo, puedes envolver un objeto que opera con metros y kilómetros con un adaptador que convierte todos los datos al sistema anglosajón (pies y millas).

¿Cómo Funciona el Adaptador?

Los adaptadores no solo convierten datos a varios formatos, sino que también ayudan a objetos con distintas interfaces a colaborar.

1 Interfaz compatible

El adaptador obtiene una interfaz compatible con uno de los objetos existentes.

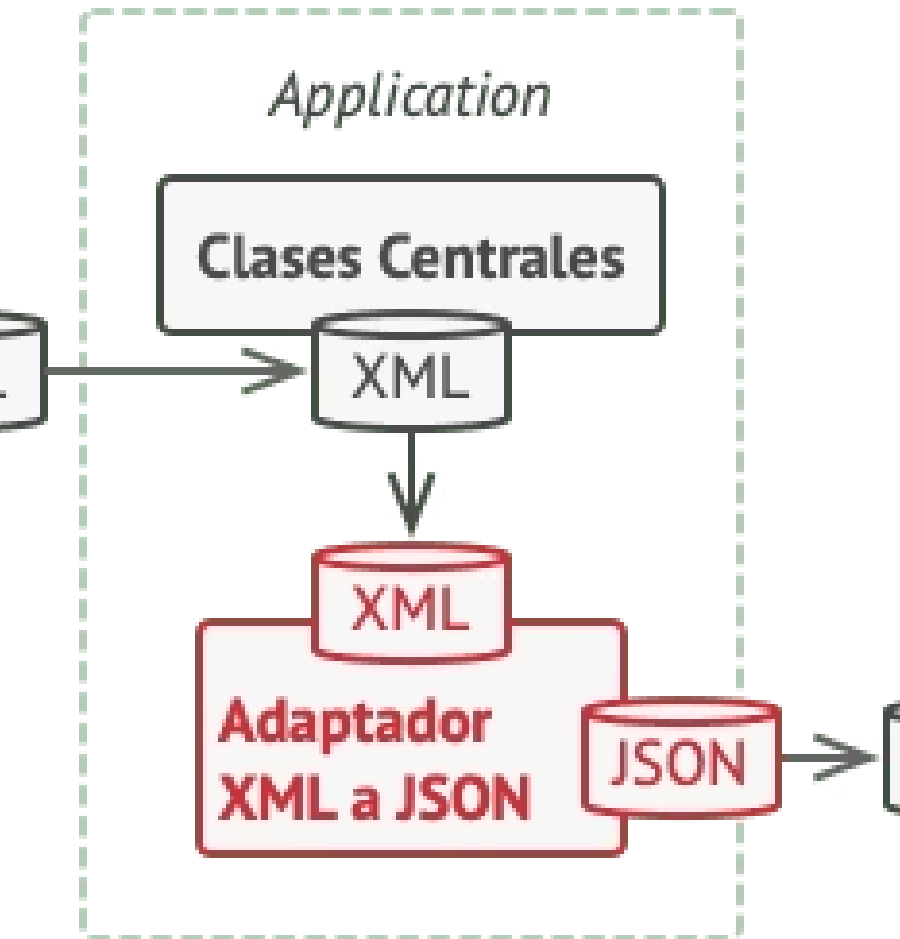
2 Invocación segura

Utilizando esta interfaz, el objeto existente puede invocar con seguridad los métodos del adaptador.

3 Traducción de la solicitud

Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, en un formato y orden que ese segundo objeto espera.

En ocasiones se puede incluso crear un **adaptador de dos direcciones** que pueda convertir las llamadas en ambos sentidos.

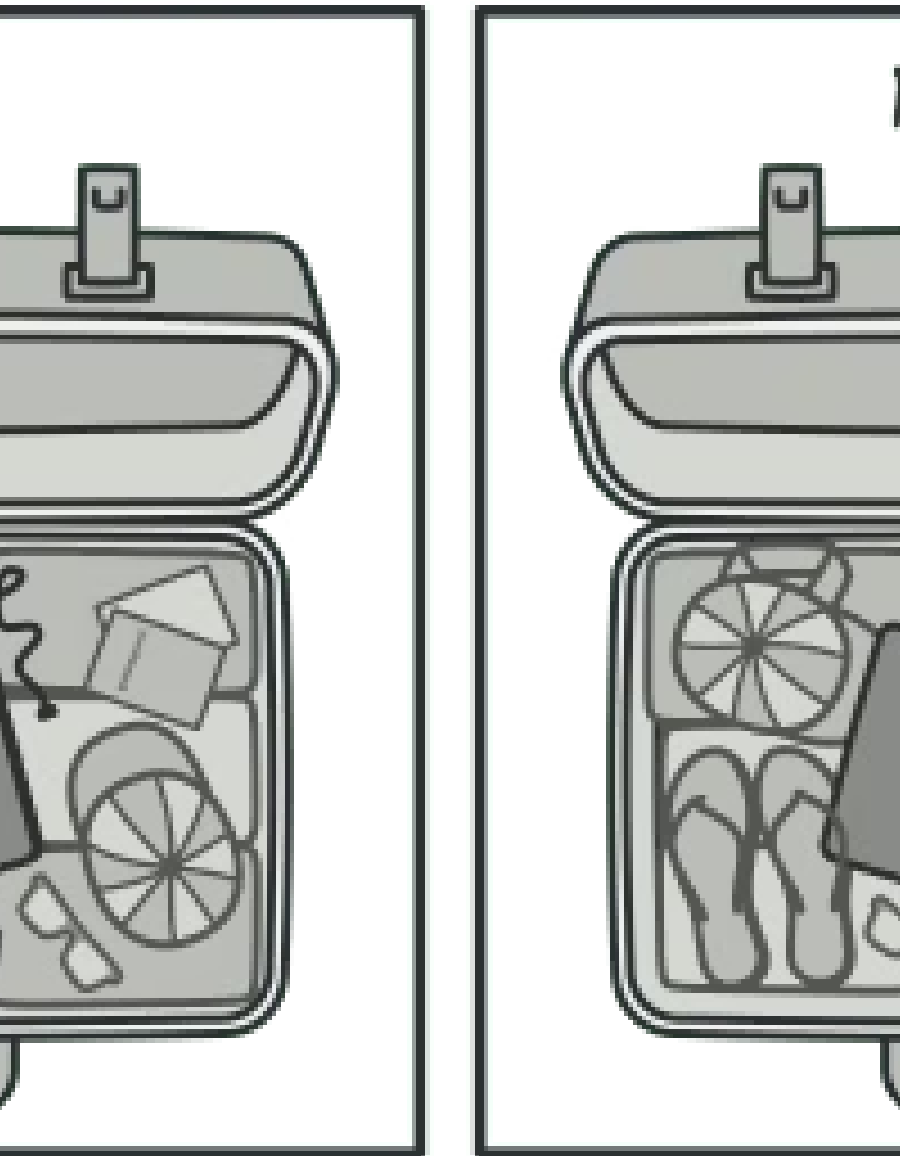


Solución Aplicada al Ejemplo

Para resolver el dilema de los formatos incompatibles, puedes crear **adaptadores de XML a JSON** para cada clase de la biblioteca de análisis con la que trabaje tu código directamente.

Después ajustas tu código para que se comuniqué con la biblioteca **únicamente a través de estos adaptadores**. Cuando un adaptador recibe una llamada, traduce los datos XML entrantes a una estructura JSON y pasa la llamada a los métodos adecuados de un objeto de análisis envuelto.

VIAJAR AL EXTRANJERO



🌐 CAPÍTULO 3

Analogía en el Mundo Real

Una maleta antes y después de un viaje al extranjero.

Cuando viajas de Europa a Estados Unidos por primera vez, puede ser que te lleves una sorpresa al intentar cargar tu computadora portátil. Los tipos de **enchufe son diferentes** en cada país, por lo que un enchufe español no sirve en Estados Unidos.

El problema puede solucionarse utilizando un **adaptador** que incluya el enchufe americano y el europeo.

Estructura del Patrón

El patrón Adapter puede implementarse de dos formas principales, cada una con sus propias características y requisitos.

Adaptador de Objetos

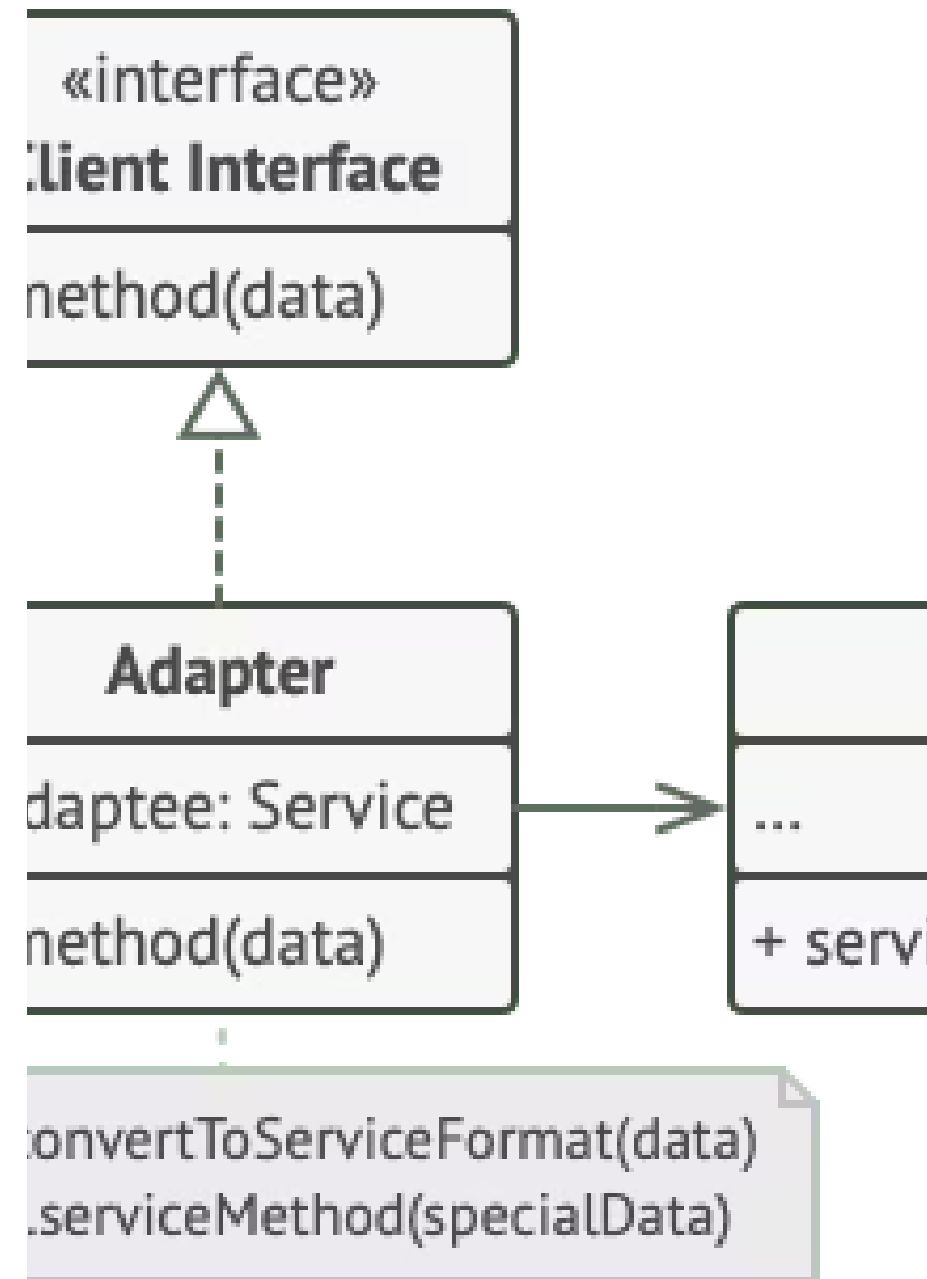
Utiliza composición de objetos. Implementable en todos los lenguajes.

Clase Adaptadora

Utiliza herencia múltiple. Solo en lenguajes como C++.

Adaptador de Objetos

Esta implementación utiliza el **principio de composición de objetos**: el adaptador implementa la interfaz de un objeto y envuelve el otro. Puede implementarse en todos los lenguajes de programación populares.



Componentes del Adaptador de Objetos

1

Cliente

Contiene la lógica de negocio existente del programa.

2

Interfaz con el Cliente

Describe un protocolo que otras clases deben seguir para colaborar con el código cliente.

3

Servicio

Clase útil (normalmente de terceros o heredada) con interfaz incompatible.

4

Clase Adaptadora

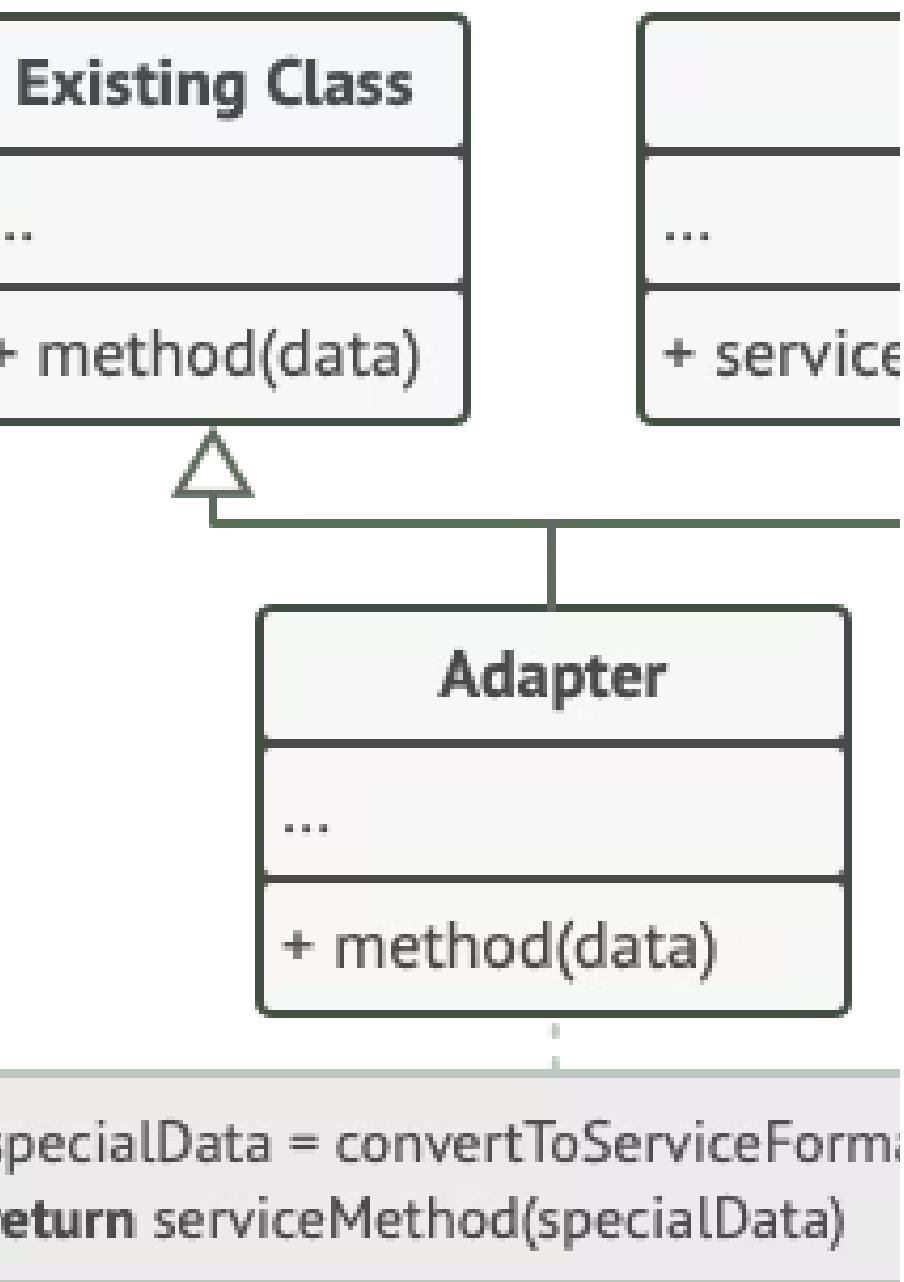
Implementa la interfaz con el cliente mientras envuelve el objeto de la clase de servicio.

Funcionamiento del Adaptador de Objetos

La clase adaptadora recibe llamadas del cliente a través de la **interfaz de cliente** y las traduce en llamadas al objeto envuelto de la clase de servicio, en un formato que pueda comprender.

- ❏ **Principio clave:** El código cliente no se acopla a la clase adaptadora concreta siempre y cuando funcione con ella a través de la interfaz con el cliente. Esto permite introducir **nuevos tipos de adaptadores** sin descomponer el código cliente existente.

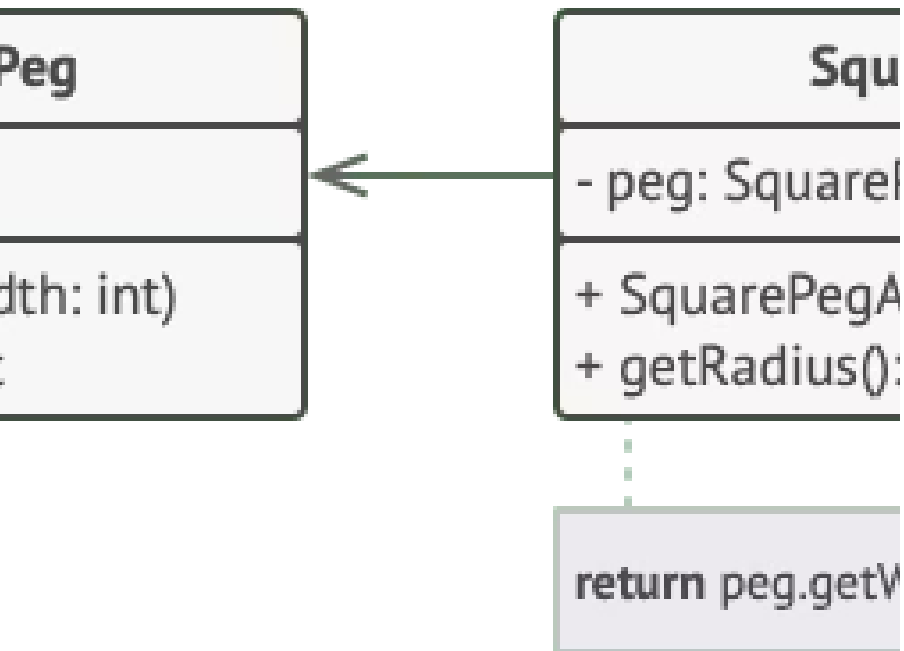
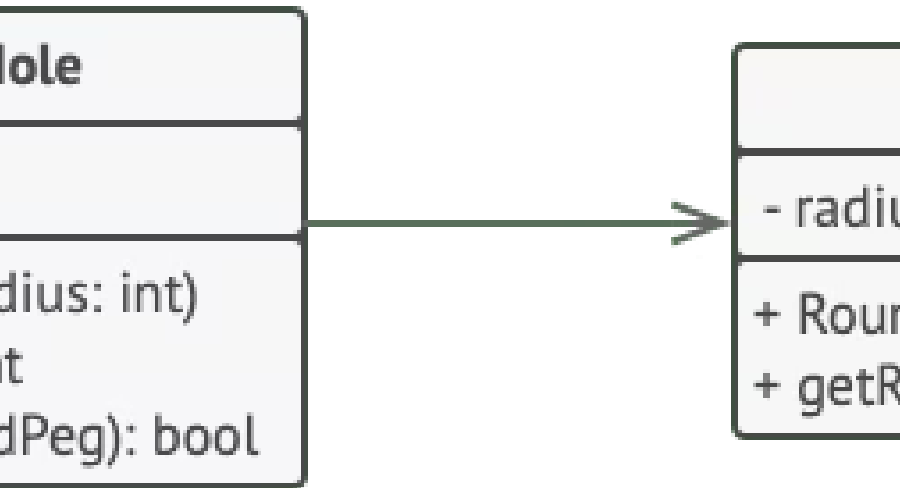
Esto resulta útil cuando la interfaz de la clase de servicio se cambia o sustituye, ya que puedes crear una nueva clase adaptadora sin cambiar el código cliente.



Clase Adaptadora (Herencia)

Esta implementación utiliza la **herencia**, porque la clase adaptadora hereda interfaces de ambos objetos al mismo tiempo. Ten en cuenta que esta opción sólo puede implementarse en lenguajes que soporten la **herencia múltiple**, como C++.

La clase adaptadora **no necesita envolver objetos** porque hereda comportamientos tanto de la clase cliente como de la clase de servicio. La adaptación tiene lugar dentro de los métodos sobrescritos.



CAPÍTULO 5

Pseudocódigo: El Ejemplo

Este ejemplo del patrón Adapter se basa en el clásico conflicto entre **piezas cuadradas** y **agujeros redondos**.

Adaptando piezas cuadradas a agujeros redondos.

El patrón Adapter finge ser una pieza redonda con un radio igual a la mitad del diámetro del cuadrado (el radio del círculo más pequeño en el que quepa la pieza cuadrada).

Pseudocódigo: Clases Base

Dos clases con interfaces compatibles: **RoundHole** (HoyoRedondo) y **RoundPeg** (PiezaRedonda).

```
// Digamos que tienes dos clases con interfaces compatibles:  
// RoundHole (HoyoRedondo) y RoundPeg (PiezaRedonda).
```

```
class RoundHole is  
  constructor RoundHole(radius) { ... }
```

```
  method getRadius() is  
    // Devuelve el radio del agujero.
```

```
  method fits(peg: RoundPeg) is  
    return this.getRadius() >= peg.getRadius()
```

```
class RoundPeg is  
  constructor RoundPeg(radius) { ... }
```

```
  method getRadius() is  
    // Devuelve el radio de la pieza.
```


Pseudocódigo: Clase Incompatible

La clase **SquarePeg** (PiezaCuadrada) tiene una interfaz incompatible con RoundHole.

```
// Pero hay una clase incompatible: SquarePeg (PiezaCuadrada).
```

```
class SquarePeg is
```

```
  constructor SquarePeg(width) { ... }
```

```
  method getWidth() is
```

```
    // Devuelve la anchura de la pieza cuadrada.
```

📌 El problema: no podemos pasar un **SquarePeg** al método `fits()` de **RoundHole** porque los tipos son incompatibles.

Pseudocódigo: La Clase Adaptadora

La clase adaptadora permite encajar piezas cuadradas en hoyos redondos extendiendo **RoundPeg**.

```
// Una clase adaptadora te permite encajar piezas cuadradas en  
// hoyos redondos. Extiende la clase RoundPeg para permitir a  
// los objetos adaptadores actuar como piezas redondas.
```

```
class SquarePegAdapter extends RoundPeg is  
  // En realidad, el adaptador contiene una instancia de la  
  // clase SquarePeg.  
  private field peg: SquarePeg
```

```
constructor SquarePegAdapter(peg: SquarePeg) is  
  this.peg = peg
```

```
method getRadius() is  
  // El adaptador simula que es una pieza redonda con un  
  // radio que pueda albergar la pieza cuadrada que el  
  // adaptador envuelve.  
  return peg.getWidth() * Math.sqrt(2) / 2
```

Pseudocódigo: Código Cliente

Así se utiliza el adaptador en el código cliente:

```
// En algún punto del código cliente.  
  
hole = new RoundHole(5)  
rpeg = new RoundPeg(5)  
hole.fits(rpeg) // verdadero  
  
small_sqpeg = new SquarePeg(5)  
large_sqpeg = new SquarePeg(10)  
hole.fits(small_sqpeg) // esto no compila (tipos incompatibles)  
  
small_sqpeg_adapter = new SquarePegAdapter(small_sqpeg)  
large_sqpeg_adapter = new SquarePegAdapter(large_sqpeg)  
hole.fits(small_sqpeg_adapter) // verdadero  
hole.fits(large_sqpeg_adapter) // falso
```

Gracias al adaptador, las piezas cuadradas ahora pueden evaluarse contra hoyos redondos sin modificar las clases originales.

Aplicabilidad

Interfaces incompatibles

Utiliza Adapter cuando quieras usar una clase existente, pero cuya interfaz no sea compatible con el resto del código. Permite crear una clase intermedia que sirva como **traductora** entre tu código y una clase heredada, de un tercero o con interfaz extraña.

Reutilizar subclases sin funcionalidad común

Utiliza el patrón cuando quieras reutilizar varias subclases existentes que carezcan de alguna funcionalidad común que no pueda añadirse a la superclase.

Aplicabilidad: Solución Elegante

Podrías extender cada subclase y colocar la funcionalidad que falta dentro de las nuevas clases hijas. No obstante, deberás **duplicar el código** en todas estas nuevas clases, lo cual huele muy mal.

Una solución mucho más elegante sería colocar la funcionalidad que falta dentro de una **clase adaptadora**. Después puedes envolver objetos a los que les falten funciones, obteniendo esas funciones necesarias de un modo dinámico.

- ❏ Para que esto funcione, las clases en cuestión deben tener una **interfaz común** y el campo de la clase adaptadora debe seguir dicha interfaz. Este procedimiento es muy similar al del patrón **Decorator**.

Cómo Implementarlo

01

Identificar clases incompatibles

Asegúrate de tener al menos dos clases con interfaces incompatibles: una clase servicio que no puedes cambiar y una o varias clases cliente.

03

Crear la clase adaptadora

Haz que siga la interfaz con el cliente. Deja todos los métodos vacíos por ahora.

05

Implementar los métodos

La clase adaptadora deberá delegar el trabajo real al objeto de servicio, gestionando la conversión de formato.

02

Declarar la interfaz con el cliente

Describe el modo en que las clases cliente se comunican con la clase de servicio.

04

Añadir referencia al servicio

Añade un campo para almacenar una referencia al objeto de servicio. Inicialízalo a través del constructor.

06

Usar a través de la interfaz

Las clases cliente deberán utilizar la clase adaptadora a través de la interfaz con el cliente.

Pros y Contras

✓ Ventajas

- **Principio de responsabilidad única.** Puedes separar la interfaz o el código de conversión de datos de la lógica de negocio primaria del programa.
- **Principio de abierto/cerrado.** Puedes introducir nuevos tipos de adaptadores al programa sin descomponer el código cliente existente, siempre y cuando trabajen con los adaptadores a través de la interfaz con el cliente.

✗ Desventajas

- La **complejidad general del código aumenta**, ya que debes introducir un grupo de nuevas interfaces y clases.
- En ocasiones resulta más sencillo **cambiar la clase de servicio** de modo que coincida con el resto de tu código.

Relaciones con Otros Patrones

Adapter vs Bridge

Bridge suele diseñarse por anticipado para desarrollar partes de forma independiente. **Adapter** se utiliza con una aplicación existente para hacer que clases incompatibles trabajen juntas.

Adapter vs Decorator

Adapter proporciona una interfaz completamente diferente. Con **Decorator** la interfaz permanece igual o se amplía. Además, Decorator admite composición recursiva, que no es posible con Adapter.

Adapter vs Proxy

Con **Adapter** se accede a un objeto a través de una interfaz diferente. Con **Proxy**, la interfaz sigue siendo la misma. Con **Decorator** se accede a través de una interfaz mejorada.

Más Relaciones con Otros Patrones

Adapter vs Facade

Facade define una nueva interfaz para objetos existentes, mientras que **Adapter** intenta hacer que la interfaz existente sea utilizable. Adapter sólo envuelve un objeto; Facade trabaja con todo un subsistema.

Patrones con estructura similar

Bridge, State, Strategy (y hasta cierto punto Adapter) tienen estructuras muy similares basadas en composición: delegar trabajo a otros objetos. Sin embargo, todos solucionan problemas diferentes.

- ❏ Un patrón no es simplemente una receta para estructurar tu código de una forma específica. También puede **comunicar a otros desarrolladores el problema que resuelve**.

Resumen: Patrón Adapter

Propósito

Permitir la colaboración entre objetos con interfaces incompatibles.

Mecanismo

Envuelve un objeto para traducir su interfaz a una compatible.

Beneficio clave

Respetar los principios de responsabilidad única y abierto/cerrado.

El patrón Adapter es una herramienta esencial cuando necesitas integrar componentes con interfaces incompatibles sin modificar el código existente.

